Software Requirements Specification (SRS)

Supply Chain Management Simulator, Team 02

Field	Value
Project	Supply Chain Management Simulator
Version	1.0
Authors	Team 02 – Rohith Karning, Rohanth Ganesha C, Rishi J Mule, Pranav Achar
Date	12-10-2026
Status	Final

Team 02

Name	SRN
Rohith Karning	PES1UG24AM229
Rohanth Ganesha C	PES1UG24AM227
Rishi J Mule	PES1UG24AM224
Pranav Achar	PES1UG24AM196

Revision History

Version	Date	Author(s)	Description of change	Approval
1.0	11-10-2026	Team 02	Initial draft	Pending
1.1	12-10-2026	Team 02	Assembled final version from all team sections	Pending

Approvals

Name	Role	Signature	Date
	Course Coordinator		
Rohith Karning	Product Owner / Integrator		
Rohanth Ganesha C	QA Lead		
Rishi J Mule	Security Test Engineer		
Pranav Achar	Dev Lead		

Table of Contents

- 1. Introduction
  - 1.1 Purpose
  - 1.2 Scope
  - 1.3 Intended Audience
  - 1.4 Definitions, Acronyms and Abbreviations
- 2. Overall Description
  - 2.1 Product Perspective
  - 2.2 Major Product Functions
  - 2.3 User Classes and Characteristics
  - 2.4 Operating Environment
  - 2.5 Design and Implementation Constraints
  - 2.6 Assumptions and Dependencies
- 3. External Interface Requirements

    - 3.1 User Interfaces
    - 3.2 Hardware Interfaces
    - 3.3 Software Interfaces
    - 3.4 Communications Interfaces
- 4. System Features (Detailed)
    - 4.1 Inventory Management
    - 4.2 Order Management
    - 4.3 Shipment and Supplier Management
    - 4.4 Analytics & Reporting
    - 4.5 Simulation Engine & CLI
- 5. Non-Functional Requirements
    - 5.1 Security Requirements
- 6. Quality Attributes & Acceptance Tests
    - 6.1 Exit Criteria
    - 6.2 Acceptance Test Suites
- 7. System Models
    - 7.1 UML Use-Case Diagrams
- 8. Requirements Traceability Matrix
    - 8.1 Functional Requirements
    - 8.2 Non-Functional Requirements
    - 8.3 Security Objectives and Security Requirements
    - 8.4 Coverage Summary

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional and security requirements of the **Supply Chain Management Simulator** (package `scm_sim`), version 1.0. The simulator models the movement of goods from suppliers to warehouses and on to customers, one simulated day at a time. It manages inventory, customer orders and replenishment shipments, and it identifies delayed shipments, stockouts and excess inventory.

The document serves three purposes:

1. It is the agreed baseline of what the system shall do, against which the design (Software Architecture Document, SAD v1.0) and the verification activities (Test Plan v1.0) are traced.
2. It assigns every requirement a fixed identifier (`SCM-F-xxx`, `SCM-NF-xxx`, `SCM-SO-x`, `SCM-SR-xxx`) and a measurable acceptance criterion (`AC-SCM-…`) linked to a test case (`TC-XXX-NN`). These identifiers are registered in `docs/requirements-registry.md` and are used unchanged in all three project documents.
3. It describes the system **as built and merged** on the `main` branch of the team repository (commit `b3702ef`), not a planned or hypothetical product.

This SRS follows the structure of IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018, adapted to the course template.

1.2 Scope

The Supply Chain Management Simulator is a standalone, command-line Python program. A user configures a scenario (suppliers, warehouses, stock policies, initial stock and scheduled customer demand), runs it for a number of days, and receives a day-by-day log plus an analytics report. All data is held in memory for the duration of a run; nothing is stored in files, databases or over a network.

In scope

Area	What the system does
Suppliers	Registers suppliers with a lead time (days) and an on-time reliability (0.0–1.0)
Warehouses	Registers warehouses with a total capacity in units across all SKUs
Inventory	Tracks stock per (warehouse, SKU); applies per-item stock policies (reorder point, reorder quantity, max level)
Customer orders	Creates orders, fulfils them in full when stock allows, back-orders them otherwise, retries back-orders oldest first, and supports cancellation
Shipments with delays	Dispatches replenishment shipments from suppliers to warehouses; delays are drawn from a seeded random generator according to supplier reliability
Automatic replenishment	Each day, dispatches one replenishment per item at or below its reorder point that has nothing already in transit, trimmed to warehouse capacity
Analytics reports	Reports delayed shipments, stockouts, low stock, excess inventory and order metrics (including fill rate) for every simulated day
CLI demo	`python -m scm_sim --days N --seed S` runs a built-in demonstration scenario and prints the daily log and the final report

Out of scope

- Graphical or web user interface
- Integration with real ERP (Enterprise Resource Planning) or WMS (Warehouse Management System) products
- Databases or any persistent storage
- Cost, pricing and profit calculations
- Demand forecasting (demand is scheduled in advance by the user)
- Multi-echelon networks (for example supplier → distribution centre → store)
- Real-time tracking of physical shipments

Benefits. The simulator lets a supply chain analyst or student observe, in a reproducible way, how lead times, supplier reliability and stock policies lead to delays, stockouts, back-orders and excess inventory, without needing real company data.

1.3 Intended Audience

Reader	How to use this document
Course Coordinator / Instructor / Evaluator	Assess completeness and quality of the requirements; use §6 and §8 to check that every requirement is tested
Supply Chain Analyst (primary user)	Understand what the simulator can and cannot model (§1.2, §2, §4)
Developers (team members)	Implement and extend modules according to §4 and §5; keep IDs consistent with the SAD and Test Plan
Testers / QA Lead	Derive and maintain test cases from the acceptance criteria in §4–§6 and the traceability matrix in §8
Security Test Engineer	Verify the security objectives and requirements in §5.1

Readers new to the project should read §1 and §2 first, then the feature section relevant to them in §4.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
SCM	Supply Chain Management: planning and controlling the flow of goods from suppliers through warehouses to customers
SKU	Stock Keeping Unit: an identifier for one distinct product, e.g. `WIDGET`, `GADGET`, `GIZMO`
Lead time	The number of days between a shipment's dispatch and its expected arrival (`Supplier.lead_time_days`)

Term	Definition
Reliability	A supplier's probability, between 0.0 and 1.0, that a shipment arrives on its expected day; a delayed shipment arrives 1–3 days late
Stockout	The state of an item whose stock level at a warehouse is exactly 0
Back-order	A customer order that could not be fulfilled in full because of insufficient stock; it has status `BACKORDERED` and is retried on later days
Reorder point	The stock level at or below which an item needs replenishment (default 20)
Reorder quantity	The number of units requested when an item is replenished (default 100)
Max level	The stock level above which an item is considered excess (default 500)
Excess inventory	Stock of an item above its max level; the amount above is the *surplus*
Fill rate	Fulfilled orders ÷ (total orders − cancelled orders), rounded to 2 decimal places; 0.0 when there are no eligible orders
Seed	The integer used to initialise the random number generator (`random.Random(seed)`); the same seed and scenario always produce the same output
Day / step	One discrete simulated day; day 0 is set-up time and the first step simulates day 1
CI	Continuous Integration: automatic execution of the test suite (GitHub Actions job `test`) on every push to `main` and every pull request
PR	Pull Request: a proposed change on a branch that must be reviewed and approved before merging into `main`
CLI	Command-Line Interface: the program is run from a terminal as `python -m scm_sim`
RTM	Requirements Traceability Matrix: the table in §8 linking each requirement to design, module and test cases
FR / NFR	Functional Requirement / Non-Functional Requirement
SO / SR	Security Objective / Security Requirement
AC	Acceptance Criterion, identified as `AC-<Req ID>`
TC	Test Case, identified as `TC-<area>-NN`
SAD	Software Architecture Document (companion document, v1.0)

2. Overall Description

2.1 Product Perspective

The Supply Chain Management Simulator is a **new, self-contained Python program**. It is not part of a larger product and does not connect to any external system. It is distributed as the Python package `scm_sim` (source in `src/scm_sim/`) and is started from a terminal.

Internally the product is organised as a layered set of modules inside one package:

Layer	Component (file)	Role
Presentation	CLI (`__main__.py`)	Parses `--days` and `--seed`, runs the demo, prints the day-by-day log and the final report
Orchestration	Simulation Engine (`simulator.py`)	Holds the scenario, advances time one day per step, and calls the domain modules in a fixed order
Domain	Inventory (`inventory.py`), Order Manager (`orders.py`), Shipment Manager (`shipments.py`)	Each owns its own state and business rules; Order Manager and Shipment Manager change stock only through the shared Inventory
Analytics	Analytics (`analytics.py`)	Pure functions that read plain data (stock snapshot, shipments, orders) and produce reports without changing any state

Development is supported by a **CI Pipeline** (`.github/workflows/ci.yml`) that runs the full pytest suite on every push to `main` and every pull request.

Goods flow through the simulated network as follows:

```
Supplier --(shipment, lead time + possible delay)--> Warehouse --
(customer order)--> Customer
```

2.2 Major Product Functions

The system shall provide the following major functions (detailed in §4):

1. **Inventory management** (§4.1): track stock per (warehouse, SKU), enforce warehouse capacity, apply per-item stock policies, suggest reorders and provide stock snapshots.
2. **Order management** (§4.2): create customer orders, fulfil them in full or back-order them, retry back-orders oldest first, cancel orders and list orders by status.
3. **Shipment and supplier management** (§4.3): register suppliers, dispatch shipments, model delays from supplier reliability using a seeded random generator, deliver arrivals into inventory and list delayed shipments.
4. **Analytics and reporting** (§4.4): detect delayed shipments, stockouts, low stock and excess inventory; compute order metrics and fill rate; build and format a daily report.
5. **Simulation engine and CLI** (§4.5): configure a scenario, advance it one day at a time in a fixed event order, replenish stock automatically, keep the daily report history, and run a built-in demo from the command line.

2.3 User Classes and Characteristics

User class	Description	Technical expertise	Main interactions
Supply Chain Analyst (primary)	Configures a scenario, runs the simulation and reads reports to find delays, stockouts and excess inventory	Basic Python and command-line use	Runs `python -m scm_sim`, or builds a `Simulation` in Python and calls `run(days)` and `final_summary()`
Instructor / Evaluator	Runs the CLI demo and the test suite to assess the system against the problem statement	Comfortable with a terminal and pytest	Runs `python -m scm_sim --days 10 --seed 42` and `pytest -v --cov=scm_sim`
Developer (team member)	Extends modules, writes tests, opens and reviews pull requests	Python 3.11, pytest, Git/GitHub	Edits `src/scm_sim/`, runs tests locally, pushes feature branches, uses CI results
System timer (secondary actor)	The daily step. Each step triggers arrivals, order fulfilment, back-order retries, replenishment and reporting	n/a (part of the system)	`Simulation.step()`

The following are **simulated entities, not human actors**: **Supplier** (sends replenishment shipments), **Warehouse** (stores stock up to its capacity) and **Customer** (places orders, identified by IDs such as `C-ACME`).

2.4 Operating Environment

Item	Requirement
Runtime	Python 3.11 or later (developed and tested with Python 3.11.9)
Operating systems	Windows 10/11, Linux and macOS
Runtime libraries	Python standard library only (`dataclasses`, `enum`, `random`, `argparse`, `typing`)
Test tools	pytest ≥ 8.0 and pytest-cov ≥ 5.0 (listed in `requirements.txt`; not needed to run the simulator)
CI environment	GitHub Actions, job `test` on `ubuntu-latest` with Python 3.11
Hardware	Any machine able to run Python 3.11; no special hardware
Network	None at runtime

2.5 Design and Implementation Constraints

1. **Standard library only at runtime.** No third-party packages may be imported by `src/scm_sim/`; pytest and pytest-cov are test-only dependencies.
2. **In-memory data.** All state (stock, policies, orders, shipments, reports) lives in Python objects for the length of one run. There is no file, database or network storage.

3. **Deterministic with a seed.** The only source of randomness is one `random.Random(seed)` instance passed to the Shipment Manager. The same seed and scenario must produce identical results on every run and machine.
4. **Discrete daily time step.** Time advances in whole days; there is no continuous or real-time clock.
5. **Package layout.** Source code lives in `src/scm_sim/` and tests in `tests/`. There is no packaging configuration (`pyproject.toml`), so the CLI is run from `src/` or with `PYTHONPATH=src`.
6. **Team and process.** The system is built by a 4-person team within one semester. Each module has one owner. All changes reach `main` only through a reviewed pull request with a passing `test` CI check.
7. **Python language level.** Code uses type hints and dataclasses compatible with Python 3.11.

2.6 Assumptions and Dependencies

Assumptions

1. Each (warehouse, SKU) item is replenished by **exactly one supplier**, assigned when its stock policy is set.
2. **Delays only make shipments later.** A shipment never arrives before its expected day; a delay adds 1–3 days.
3. **Customer demand is known in advance.** Orders are scheduled for specific future days before or during the run; the system does not forecast demand.
4. Orders are fulfilled **in full or not at all** (no partial shipments to customers).
5. Suppliers have unlimited stock; only lead time and reliability limit them.
6. Stock moves only between a supplier and a warehouse, and from a warehouse to a customer; there are no transfers between warehouses.
7. Capacity is measured in units summed over all SKUs in a warehouse; all units take the same space.

Dependencies

1. A Python 3.11+ interpreter on the user's machine.
2. pytest and pytest-cov for running the automated tests.
3. GitHub and GitHub Actions for version control, code review and CI.
4. The companion documents: SAD v1.0 (design) and Test Plan v1.0 (verification), and the requirements registry `docs/requirements-registry.md`.

3. External Interface Requirements

3.1 User Interfaces

The system provides a command-line interface (CLI) as its sole user-facing interface. There is no graphical user interface (GUI) or web interface.

1. **Invocation and Command-Line Arguments:**
   - The user executes the simulator via the standard Python module entry point:

     ```
     python -m scm_sim [--days N] [--seed S]
     ```

   - **`--days N`:** The duration of the simulation run in simulated days (integer). If omitted, the demo defaults to 10 days. The system shall validate that `N >= 1`; if `N < 1`, the CLI shall display a standard usage error message and terminate with exit code 2.
   - **`--seed S`:** The integer seed for the pseudorandom number generator (defaults to 42). Enables deterministic reproducibility across multiple runs.
   - **`--help` / `-h`:** Displays usage syntax, available argument flags, and default parameter values, terminating with exit code 0.

2. **Output Formatting and Execution Log:**
   - During simulation execution, the system shall stream a day-by-day progress line for each simulated day step to standard output (`stdout`), detailing day index, arriving shipments, orders placed, fulfillments, and replenishments dispatched.
   - Upon completing the specified duration, the system shall render a multi-line formatted summary report (`=== Supply Chain Report: Day N ===`) containing itemized counts for:
     - **Delayed shipments:** Count and item details (shipment ID, SKU, destination warehouse, and days late).
     - **Stockouts:** Count and item details (warehouse ID and SKU with stock level equal to 0).
     - **Low stock:** Count and item details (warehouse ID, SKU, and remaining stock level at or below reorder point).
     - **Excess:** Count and item details (warehouse ID, SKU, total quantity, and surplus units exceeding maximum level).
     - **Orders:** Total orders, counts categorized by status (`pending`, `fulfilled`, `backordered`, `cancelled`), and calculated aggregate fill rate rounded to two decimal places.
   - If any reporting category contains zero problem instances, the system shall explicitly print `No issues` under that category header.
   - On successful completion of the run, the process shall terminate with exit code 0.

3.2 Hardware Interfaces

The system requires no specialized hardware interfaces or external physical device controllers. The simulator executes entirely in user-space memory on standard computing hardware: - **Processor:** Standard x86-64 or ARM64 architecture processor. - **Memory (RAM):** Minimum 512 MB available physical RAM (in-memory footprint is negligible, consuming < 50 MB during a 365-day simulation). - **Storage:** Minimum 100 MB free disk space for repository files, Python environment, and test caches. - **Input/Output Devices:** Standard keyboard for CLI input and terminal display / console standard output (`stdout` / `stderr`) for text rendering.

3.3 Software Interfaces

The system interfaces strictly with standard local software components and development tools: - **Operating System:** Platform-independent execution supporting Windows 10/11, Ubuntu Linux (20.04+), and macOS (12+). - **Python Runtime:** Python 3.11 or higher runtime interpreter (`python >= 3.11.0`). The system relies strictly on standard library modules (`dataclasses`, `enum`, `typing`, `random`, `argparse`, `sys`). - **Test Automation Framework:** `pytest` (version >= 8.0) and `pytest-cov` (version >= 5.0) for executing unit, integration, and system test suites and computing statement coverage metrics. - **Continuous Integration Service:** GitHub Actions hosted runners (`ubuntu-latest`) executing the `.github/workflows/ci.yml` workflow on every pull request and push to `main`. - **Data Persistence:** None. All entities (inventories, orders, shipments, policies) reside in in-memory Python data structures (`dict`, `list`). No external database management systems (DBMS) or file storage drivers are utilized.

3.4 Communications Interfaces

- **Runtime Communications:** None. The application operates strictly as an isolated, standalone local process. No network protocols, sockets, HTTP servers, REST APIs, or remote procedure calls (RPC) are opened or consumed at runtime.
- **Development Communications:** Git version control over HTTPS (`https://github.com/rohithkarning-debug/supply-chain-simulator.git`) and GitHub remote APIs for pull requests, automated CI dispatch, code reviews, and issue tracking during project development.

4. System Features (Detailed)

Each requirement below has an acceptance criterion and a reference test case. IDs follow SCM-F-###.

4.1 Inventory Management

Description: The Inventory component (`scm_sim.inventory`, class `Inventory`) is the single source of truth for stock. It records the quantity of every SKU held at every warehouse, enforces each registered warehouse's total capacity, stores a stock policy (reorder point, reorder quantity, max level) per (warehouse, SKU) item, and tells the rest of the system which items need replenishment or hold excess stock. The Order Manager and the Shipment Manager change stock only through this component, and the Analytics component reads it only through an independent snapshot.

Priority: High. Every other feature depends on correct stock levels.

Stimulus / response sequences

Stimulus	Response
A shipment arrives at `WH-NORTH` with 150 `WIDGET`	Stock of (`WH-NORTH`, `WIDGET`) increases by 150, unless that would exceed the warehouse capacity
A customer order for 30 `WIDGET` is fulfilled from `WH-NORTH`	Stock of (`WH-NORTH`, `WIDGET`) decreases by 30
The daily replenishment check runs	Inventory returns every tracked item at or below its reorder point together with its reorder quantity
The daily report is built	Inventory returns a copy of all stock levels

Functional requirements

Req ID	Requirement (The system shall…)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SCM-F-001	The system shall track the stock level of every (warehouse, SKU) pair, returning 0 for a pair that has never been stocked. It shall add and remove whole units, rejecting any quantity ≤ 0 with `ValueError`, and shall reject a removal larger than the current level with `InsufficientStockError`, leaving the stock level unchanged.	Functional	High	Problem statement ("manage inventory"); Supply Chain Analyst	**AC-SCM-F-001:** Given an empty inventory, `level("WH-NORTH", "WIDGET")` returns 0. After `add("WH-NORTH", "WIDGET", 100)` it returns 100. After `remove("WH-NORTH", "WIDGET", 30)` it returns 70. `remove("WH-NORTH", "WIDGET", 71)` raises `InsufficientStockError` and the level stays 70. `add(…, 0)` and `remove(…, 0)` raise `ValueError`. `is_stockout` is True exactly when the level is 0. **Test: TC-INV-01**	Methods `level`, `add`, `remove`, `is_stockout`. `add` and `remove` return the new level. Used by Order Manager (`remove`) and Shipment Manager (`add`).
SCM-F-002	The system shall allow a warehouse to be registered with a total capacity greater than 0 (otherwise `ValueError`). It shall reject with `CapacityExceededError` any addition that would make the warehouse's total stock across all SKUs exceed that capacity, leaving stock unchanged. Warehouses that are never registered shall	Functional	High	Problem statement ("warehouses"); Supply Chain Analyst	**AC-SCM-F-002:** With `register_warehouse("WH-SOUTH", 500)` and 400 units already stored, adding 100 more succeeds (total exactly 500) and adding 1 further unit raises `CapacityExceededError` with the total still 500. `register_warehouse("WH-SOUTH", 0)` raises `ValueError`. An unregistered warehouse accepts any positive	Methods `register_warehouse`, `total_in_warehouse`, `add`. Capacity is checked on `add` only: registering a capacity smaller than the stock already held is not rejected.

Req ID	Requirement (The system shall…)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	have unlimited capacity.				quantity. `total_in_warehouse` returns the sum over all SKUs. **Test: TC-INV-01**	
SCM-F-003	The system shall store a stock policy per (warehouse, SKU) with a reorder point, reorder quantity and max level (defaults 20, 100 and 500), rejecting negative values or a max level below the reorder point with `ValueError`. It shall report that an item needs a reorder when its level ≤ its reorder point, and that it holds excess stock when its level > its max level.	Functional	High	Problem statement ("stockouts and excess inventory"); Supply Chain Analyst	**AC-SCM-F-003:** `StockPolicy()` has `reorder_point=20`, `reorder_quantity=100`, `max_level=500`. `StockPolicy(reorder_point=-1)` and `StockPolicy(reorder_point=30, max_level=20)` raise `ValueError`; `max_level == reorder_point` is accepted. With the default policy, levels 20 and 21 give `needs_reorder` True and False respectively, and levels 500 and 501 give `is_excess` False and True respectively. An item with no policy set uses the defaults. **Test: TC-INV-02**	`StockPolicy` is a frozen (immutable) dataclass. Methods `set_policy`, `get_policy`, `needs_reorder`, `is_excess`. Policies are per item, while Analytics reports use one global threshold (SAD §4.6).
SCM-F-004	The system shall return a list of `(warehouse, sku, reorder_quantity)` for every tracked item (one that has ever been stocked or has a policy set) at or below its reorder point, sorted by warehouse and then SKU. It shall also return a snapshot of all stock levels as a new dictionary `{(warehouse, sku): qty}` whose modification does not affect the inventory.	Functional	Medium	Simulation Engine (automatic replenishment); Analytics (reports)	**AC-SCM-F-004:** For items (`WH-SOUTH`, `GADGET`) at 10 and (`WH-NORTH`, `WIDGET`) at 5, both with default policies, plus (`WH-NORTH`, `GIZMO`) at 600, `reorder_suggestions()` returns `[("WH-NORTH", "WIDGET", 100), ("WH-SOUTH", "GADGET", 100)]` in that order. A new inventory returns `[]`. Changing a value in the dictionary returned by `snapshot()` leaves `level()` unchanged. **Test: TC-INV-02**	Methods `reorder_suggestions`, `snapshot`. The Simulation Engine uses `reorder_suggestions` in step 4 of each day (SCM-F-042); Analytics reads `snapshot` (SCM-F-031).

4.2 Order Management

Description: The Order Management subsystem (`scm_sim.orders`) is responsible for managing customer demand and tracking the end-to-end lifecycle of customer orders within the supply chain simulator. It encapsulates order creation, validation of demand quantities, deterministic sequential identification (`ORD-0001`, `ORD-0002`, …), status progression across `PENDING`, `FULFILLED`, `BACKORDERED`, and `CANCELLED`, atomic full-order fulfillment against warehouse inventory without partial shipments, prioritized batch retry of open orders on a First-In-First-Out (FIFO) basis, order retrieval by identifier, and state-guarded order cancellation.

Req ID	Requirement	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Case Ref	Comments / Dependencies
SCM-F-010	The system shall create customer orders containing customer identifier, warehouse identifier, SKU identifier, order quantity, and	Functional	High	Supply Chain Analyst	AC-SCM-F-010: Successful order creation assigns sequential identifiers `ORD-0001`, `ORD-0002`, … with status `PENDING` and records	Managed by `OrderManager.create()` in `scm_sim.orders`. Initial state before any fulfillment attempt.

Req ID	Requirement	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Case Ref	Comments / Dependencies
	creation day, generating sequential order identifiers prefixed with `ORD-` formatted as four zero-padded digits starting from `ORD-0001`, setting initial status to `PENDING`, and rejecting any order creation request where quantity is less than or equal to 0 with a `ValueError`.				`created_day`; order creation with quantity 0 raises `ValueError` before state changes. Test: TC-ORD-01	
SCM-F-011	The system shall attempt to fulfil an order in full by querying available stock at the specified warehouse via `Inventory.level(warehouse, sku)` and deducting stock via `Inventory.remove(warehouse, sku, qty)`; if available stock is greater than or equal to the order quantity, the system shall deduct the quantity from inventory, transition the order status to `FULFILLED`, and record the current day as `fulfilled_day`; if available stock is less than the order quantity, the system shall transition the order status to `BACKORDERED` and leave inventory levels unchanged without partial fulfillment; orders already in `FULFILLED` or `CANCELLED` status shall remain unchanged and return `False`.	Functional	High	Supply Chain Analyst	AC-SCM-F-011: When warehouse inventory order quantity, stock is decremented by quantity, order status transitions to `FULFILLED`, and `fulfilled_day` equals current day; when stock < quantity, status transitions to `BACKORDERED` and inventory is unaltered; already `FULFILLED` or `CANCELLED` orders are unmodified. Test: TC-ORD-01	Managed by `OrderManager.try_fulfil()`. Depends on `scm_sim.inventory.Inventory`.
SCM-F-012	The system shall retry fulfillment for all open orders currently in `PENDING` or `BACKORDERED` status upon invocation of `process_open(day)`, evaluating orders in strict oldest-first priority sorted ascending by `created_day` with `order_id` in ascending alphanumeric order as tie-breaker, and return a list containing all orders newly transitioned to `FULFILLED` during the retry pass.	Functional	High	Supply Chain Analyst	AC-SCM-F-012: Calling `process_open(day)` evaluates all `PENDING` and `BACKORDERED` orders sorted by `(created_day, order_id)`; orders with sufficient stock transition to `FULFILLED`, decrement inventory, and appear in the returned list; orders with insufficient stock remain `BACKORDERED`. Test: TC-ORD-02	Invoked in step (3) of daily simulation loop `Simulation.step()`. Enforces FIFO fairness across competing orders.
SCM-F-013	The system shall support order cancellation by transitioning the order status to `CANCELLED` unless the order is	Functional	Medium	Supply Chain Analyst	AC-SCM-F-013: Cancelling a `PENDING` or `BACKORDERED` order sets status to `CANCELLED`; cancelling a	Managed by `OrderManager.cancel()`, `OrderManager.get()`, and

Req ID	Requirement	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Case Ref	Comments / Dependencies
	currently in FULFILLED status, in which case the system shall raise a ValueError; the system shall retrieve orders by identifier via get(order_id), raising KeyError if the identifier is unrecognized, and filter orders by status via by_status(status).				FULFILLED order raises ValueError; querying an unknown order_id raises KeyError; by_status(status) returns all orders matching the requested OrderStatus. Test: TC-ORD-02	OrderManager.by_status().

4.3 Shipment and Supplier Management

Description: The Shipment and Supplier Management subsystem models external supplier entities, schedules and dispatches goods replenishments to network warehouses, simulates stochastic lead-time transit delays governed by supplier reliability metrics, receives completed shipments into warehouse inventory, and provides real-time querying for in-transit and overdue stock movements.

Req ID	Requirement (The system shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SCM-F-020	The system shall allow registering suppliers with a unique alphanumeric identifier (supplier_id), a name string, a baseline lead time in days (lead_time_days, default 2), and an on-time delivery reliability factor (reliability, default 1.0), and shall raise a ValueError without altering state if lead_time_days < 0, if reliability is outside [0.0, 1.0], or if a supplier with the specified supplier_id is already registered.	Functional	High	Supply Chain Analyst / Simulation Engine	AC-SCM-F-020: Valid parameters construct a Supplier dataclass and register it in ShipmentManager._suppliers; registering an existing supplier_id (e.g., duplicate SUP-RELIABLE) raises ValueError; lead_time_days < 0 or reliability not in [0.0, 1.0] raises ValueError. Test: TC-SHP-01	Component: Supplier, ShipmentManager.add_supplier in scm_sim.shipments. Enforces input validation and entity uniqueness (SCM-SR-001, SCM-SR-004).
SCM-F-021	The system shall dispatch a replenishment shipment from a registered supplier to a designated warehouse for a given SKU and quantity (quantity > 0) on a specified simulation day, assigning sequential unique identifiers formatted as SHP-0001, SHP-0002, ..., computing the scheduled arrival day as expected_day = day + supplier.lead_time_days, setting initial status to ShipmentStatus.IN_TRANSIT, and shall raise a KeyError if supplier_id is unknown or a	Functional	High	Supply Chain Analyst / System timer	AC-SCM-F-021: Calling ShipmentManager.dispatch() creates a Shipment with status IN_TRANSIT, sets expected_day = day + lead_time_days, assigns sequential IDs matching ^SHP-\d{4}$, raises KeyError when dispatched with an unknown supplier ID (e.g., SUP-UNKNOWN), and raises ValueError when quantity <= 0. Test: TC-SHP-01	Component: ShipmentManager.dispatch, Shipment in scm_sim.shipments. Triggered during daily simulation step 4 (replenishment). Depends on SCM-F-020.

Req ID	Requirement (The system shall…)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	`ValueError` if `quantity <= 0` without mutating state.					
SCM-F-022	The system shall model stochastic delivery delays during shipment dispatch such that if an explicit integer `delay` is provided, `arrival_day` is set to `expected_day + delay`; and if `delay` is `None`, a random draw from an injected `random.Random` instance is evaluated against `supplier.reliability`, assigning `delay = rng.randint(1, 3)` if `rng.random() > supplier.reliability` and `0` otherwise, guaranteeing that suppliers with `reliability == 1.0` are never delayed (`delay == 0`).	Functional	High	Supply Chain Analyst / Instructor	AC-SCM-F-022: When `delay=None` and `supplier.reliability == 1.0` (e.g., `SUP-RELIABLE`), `arrival_day` strictly equals `expected_day`; for suppliers with `reliability < 1.0` (e.g., `SUP-FLAKY`), delays of 1 to 3 days occur whenever `rng.random() > reliability`; passing an explicit `delay` value overrides random sampling deterministically. Test: TC-SHP-02	Component: `ShipmentManager.dispatch` in `scm_sim.shipments`. Supports deterministic reproducibility via seeded PRNG (SCM-NF-002).
SCM-F-023	The system shall, upon daily arrival processing (`process_arrivals(day)`), deliver every shipment in status `IN_TRANSIT` whose `arrival_day <= day` by incrementing warehouse stock via `inventory.add(warehouse_id, sku, quantity)`, transitioning status to `ShipmentStatus.DELIVERED`, and setting `actual_day = day`; and shall provide query methods `in_transit()` returning all active in-transit shipments and `delayed(current_day)` returning all shipments where `days_late(current_day) > 0`, where `days_late` evaluates to `max(0, actual_day - expected_day)` if delivered, or `max(0, current_day - expected_day)` if still in transit.	Functional	High	System timer / Supply Chain Analyst	AC-SCM-F-023: Calling `ShipmentManager.process_arrivals(day)` updates inventory stock solely via `inventory.add()`, updates status to `DELIVERED`, sets `actual_day = day`, and returns delivered shipments; shipments with `arrival_day > day` remain `IN_TRANSIT` with inventory unchanged; `delayed(current_day)` accurately flags both in-transit late and delivered late shipments. Test: TC-SHP-02	Component: `ShipmentManager.process_arrivals`, `in_transit`, `delayed`, `Shipment.days_late` in `scm_sim.shipments`. Interacts with `Inventory.add` (SCM-F-001, SCM-F-002).

4.4 Analytics & Reporting

Description: The Analytics and Reporting subsystem derives operational insight from the simulator's plain state snapshots without mutating them. It detects late inbound shipments by comparing expected arrival days against the current or actual delivery day, classifies inventory health into stockouts, low stock and excess inventory

against configurable thresholds, aggregates customer order counts and a fill rate, and composes these findings into a single daily report that is rendered as human-readable text for the Supply Chain Analyst. All operations are implemented as pure functions in `scm_sim.analytics` that hold no internal state, depend on no other component, and are invoked once per simulated day by the Simulation Engine as the final step of the daily time-step.

Req ID	Requirement (The system shall…)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SCM-F-030	The system shall identify late inbound shipments by computing `days_late` for each shipment as `(actual_day if the shipment has been delivered else current_day) − expected_day`, shall include a shipment in the delayed list if and only if `days_late > 0`, shall return for each delayed shipment the keys `shipment_id`, `sku`, `warehouse_id` and `days_late`, and shall order the returned list by `days_late` descending, breaking ties by ascending `shipment_id`.	Functional	High	Supply Chain Analyst / Instructor / Evaluator	AC-SCM-F-030: For an in-transit `SHP-0001` with `expected_day = 5` evaluated on `current_day = 8`, `find_delayed_shipments` returns exactly one entry with `days_late = 3`; for the same shipment delivered with `actual_day = 7` and evaluated on `current_day = 20`, `days_late = 2` (the recorded `actual_day`, not the current day, is used); a shipment arriving on its expected day (`actual_day = 5`) and one arriving early (`actual_day = 3`) are both excluded; given two shipments 1 and 4 days late, the returned `shipment_id` order is most-late-first. Test: TC-ANA-01	Component: `find_delayed_shipments` in `scm_sim.analytics`. Reads `Shipment.expected_day` and `Shipment.actual_day` produced by the Shipment Manager (SCM-F-021, SCM-F-022, SCM-F-023). Supports deterministic reporting via the seeded PRNG (SCM-NF-002).
SCM-F-031	The system shall classify every tracked item in the stock snapshot into exactly one of three exception classes and shall return each class as a deterministically sorted list: (a) stockouts, being items with quantity exactly equal to 0, returned as `(warehouse_id, sku)`; (b) low stock, being items with quantity satisfying `reorder_point`, returned as `(warehouse_id, sku, qty)`; and (c) excess inventory, being items with quantity strictly greater than `max_level`, returned as `(warehouse_id, sku, qty, surplus)` where ; the parameters `reorder_point` and `max_level` shall default to 20 and 500 respectively and shall be overridable by the caller on each call.	Functional	High	Supply Chain Analyst / Instructor / Evaluator	AC-SCM-F-031: For the stock snapshot `{("WH-NORTH","WIDGET"): 0, ("WH-NORTH","GADGET"): 1, ("WH-NORTH","GIZMO"): 20, ("WH-NORTH","BOLT"): 21, ("WH-SOUTH","WIDGET"): 500, ("WH-SOUTH","GADGET"): 501, ("WH-SOUTH","GIZMO"): 0}` with default thresholds, `find_stockouts` returns `[("WH-NORTH","WIDGET"), ("WH-SOUTH","GIZMO")]`, `find_low_stock` returns `[("WH-NORTH","GADGET",1), ("WH-NORTH","GIZMO",20)]` (0 excluded, 20 included, 21 excluded) and `find_excess` returns `[("WH-SOUTH","GADGET",501,1)]` (500 excluded, 501 included); passing	Components: `find_stockouts`, `find_low_stock`, `find_excess` in `scm_sim.analytics`. Operate on the independent snapshot returned by `Inventory.snapshot()` (SCM-F-004, SCM-SR-007). The default thresholds match the default `StockPolicy(reorder_point=20, max_level=500)` (SCM-F-003); the known limitation that Analytics applies one global threshold while Inventory holds per-item policies is recorded in SAD §4.6.

Req ID	Requirement (The system shall…)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
					`reorder_point = 21` additionally reports `("WH-NORTH","BOLT",21)` and passing `max_level = 100` for a quantity of 120 reports `(…, 120, 20)`. Test: TC-ANA-01	
SCM-F-032	The system shall aggregate order statistics over a list of orders by returning the keys `total`, `pending`, `fulfilled`, `backordered`, `cancelled` and `fill_rate`, where each status count reflects the number of orders in the corresponding `OrderStatus` and `total` equals the length of the supplied list; the system shall compute the fill rate as , so that cancelled orders are excluded from the denominator, and shall return `fill_rate = 0.0` when `total − cancelled` is not greater than 0; the system shall accept order status values supplied either as `OrderStatus` enumeration members or as equivalent plain strings, including mixed usage within one list.	Functional	Medium	Supply Chain Analyst / Instructor / Evaluator	AC-SCM-F-032: For five orders with statuses PENDING, FULFILLED, FULFILLED, BACKORDERED and CANCELLED, `order_metrics` returns `total = 5`, `pending = 1`, `fulfilled = 2`, `backordered = 1`, `cancelled = 1`; for orders FULFILLED, FULFILLED, PENDING, CANCELLED the fill rate is (2 decimal places); for an empty order list the result is `total = 0` and `fill_rate = 0.0`; a list whose statuses are supplied as an `OrderStatus` member for one order and the plain string `"FULFILLED"` for another yields `fulfilled = 2`. Test: TC-ANA-02	Component: `order_metrics` in `scm_sim.analytics`. Consumes the `Order` objects produced by the Order Manager (SCM-F-010, SCM-F-011, SCM-F-012, SCM-F-013). No division-by-zero error occurs when every order is cancelled or the list is empty.
SCM-F-033	The system shall compose the findings of SCM-F-030, SCM-F-031 and SCM-F-032 into a single daily report dictionary that contains exactly the keys `day`, `delayed_shipments`, `stockouts`, `low_stock`, `excess` and `orders`, where `day` is the simulation day supplied by the caller and `orders` holds the output of `order_metrics`, and shall forward any caller-supplied `reorder_point` and `max_level` thresholds to the underlying stock classification; the system shall render a report dictionary as a multi-line text block whose first line is `=== Supply Chain Report: Day N ===`, which prints one titled section per report key with its item count in the	Functional	Medium	Supply Chain Analyst / Instructor / Evaluator	AC-SCM-F-033: `build_report(stock, shipments, orders, current_day = 8)` returns a dictionary whose key set is exactly `{day, delayed_shipments, stockouts, low_stock, excess, orders}` with `day = 8` and `orders.fill_rate` populated; `format_report` of that report contains the strings `Day 8`, `Delayed shipments (1)`, `Stockouts (1)`, `Low stock (1)`, `Excess (1)` and `Orders (2)`; `format_report` of a report built from an empty snapshot and empty lists contains `Delayed shipments (0)` and the text `No issues` exactly four times, once per empty section. Test:	Components: `build_report`, `format_report` in `scm_sim.analytics`. `build_report` is invoked as step 5 of `Simulation.step()` and every returned report is retained by the Simulation Engine (SCM-F-041). `format_report` supplies the final summary printed by the CLI (SCM-F-043) and satisfies the empty-section reporting rule of SCM-NF-005.

Req ID	Requirement (The system shall…)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	form `Section name (count)`, and which prints the literal text `No issues` in place of item listings for every empty section.				TC-ANA-02	

4.5 Simulation Engine & CLI

Description: The Simulation Engine (`scm_sim.simulator`, class `Simulation`) wires the Inventory, Order Manager, Shipment Manager and Analytics components into one discrete, day-by-day simulation. The user (or the built-in `build_demo()` scenario) configures suppliers, warehouses, stock policies, initial stock and future customer demand. Each call to `step()` then advances the clock by exactly one day and runs a fixed sequence of events, automatically replenishing low stock and recording an analytics report. The CLI (`scm_sim.__main__`, function `main`) exposes the demo scenario as `python -m scm_sim --days N --seed S`.

Priority: High (simulation engine), Medium (CLI).

Stimulus / response sequences

Stimulus	Response
Analyst adds suppliers, warehouses, policies, stock and scheduled orders	The scenario is stored at day 0; invalid references or values are rejected before anything is stored
Analyst or System timer calls `step()`	The day counter increases by 1 and the five daily events run in order; that day's report is returned and appended to `history`
Analyst calls `run(10)`	Ten steps run; a list of ten reports is returned
Evaluator runs `python -m scm_sim --days 10 --seed 42`	The demo prints a log for days 1–10 and the formatted final report, then exits with code 0

Daily event order (one call to `step()`): 1. Deliver shipments arriving today into inventory. 2. Create today's scheduled customer orders and try to fulfil each (otherwise back-order it). 3. Retry all open back-orders, oldest first. 4. Dispatch one replenishment per item that needs a reorder and has nothing already in transit, trimmed to warehouse capacity. 5. Build that day's analytics report and append it to the history.

Functional requirements

Req ID	Requirement (The system shall…)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SCM-F-040	The system shall let a user configure a scenario before or between steps: register suppliers (ID, name, lead time, reliability) and warehouses with a capacity; set a per-item stock policy linked to one registered supplier; add initial stock; and schedule customer orders for future days. It shall reject an unknown warehouse or supplier with `KeyError`, an order quantity ≤ 0 with `ValueError`, and an order for a day that is not later than the current day with `ValueError`, without storing anything.	Functional	High	Problem statement ("suppliers, warehouses, customers"); Supply Chain Analyst	**AC-SCM-F-040:** On a new `Simulation` with warehouse `WH` registered: `schedule_order(1, "C-ACME", "NOWHERE", "WIDGET", 5)` raises `KeyError`; `schedule_order(1, "C-ACME", "WH", "WIDGET", 0)` raises `ValueError`; after one `step()`, `schedule_order(1, "C-ACME", "WH", "WIDGET", 5)` raises `ValueError` because day 1 has already been simulated. `set_policy("WH", "WIDGET", "SUP-UNKNOWN")` raises `KeyError`. **Test: TC-SIM-**	Methods `add_supplier`, `add_warehouse`, `set_policy`, `add_stock`, `schedule_order`. Validation of values is delegated to `Supplier`, `Inventory.register_warehouse`, `StockPolicy` and `Inventory.add`. SKUs are not validated: an order for a SKU never stocked is back-ordered.

Req ID	Requirement (The system shall…)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
					02	
SCM-F-041	The system shall advance the simulation one day per `step()` call, starting from day 0, running in this fixed order: (1) deliver arrivals, (2) create and fulfil today's orders, (3) retry back-orders, (4) replenish, (5) build the report. It shall return that day's report and keep every daily report in order in `history`; `run(days)` shall perform `days` steps (`ValueError` if negative) and return their reports.	Functional	High	Problem statement ("simulate movement of goods"); Supply Chain Analyst, System timer	**AC-SCM-F-041:** A new simulation has `day == 0`. After `step()`, `day == 1` and the report has `"day": 1`; after `run(3)`, `day == 4` and the `history` days are `[1, 2, 3, 4]`. With 30 units in stock, a 60-unit order on day 1 is `BACKORDERED`; when the replenishment arrives on day 4, the back-order becomes `FULFILLED` with `fulfilled_day == 4` on that same day, because arrivals are processed before back-orders are retried. **Test: TC-SIM-01**	Method `step`, `run`, attribute `history`. One `DayLog` per day is kept in `logs` (delivered, fulfilled, back-ordered, back-orders filled, reorders). Today's new orders are served before older back-orders (SAD §4.6).
SCM-F-042	The system shall, in step 4 of every day, dispatch exactly one replenishment shipment from the item's assigned supplier for each item that `Inventory.reorder_suggestions()` reports and that has no shipment already in transit, for its reorder quantity reduced so that on-hand stock plus all stock in transit to that warehouse never exceeds the warehouse capacity. If no capacity remains, or the item has no assigned supplier, no shipment shall be dispatched.	Functional	High	Problem statement ("manage shipments"); Supply Chain Analyst	**AC-SCM-F-042:** With 10 units on hand, reorder point 20, reorder quantity 100 and lead time 2: day 1 dispatches exactly 1 shipment; day 2 dispatches 0 (one is in transit); on day 3 it arrives, stock is 110 and the total number of reorders is still 1. In a warehouse of capacity 50 holding 10 units with reorder quantity 100, the shipment is trimmed to 40 and after 3 days stock is exactly 50 with no `CapacityExceededError`. **Test: TC-SIM-01**	Private method `_replenish`. Uses `Inventory.reorder_suggestions`, `Inventory.total_in_warehouse`, `ShipmentManager.in_transit` and `ShipmentManager.dispatch`. Trimming guarantees arrivals fit, because stock only rises through arrivals.
SCM-F-043	The system shall provide a command-line interface `python -m scm_sim [--days N] [--seed S]` (defaults 10 and 42) that runs the demo scenario from `build_demo(seed)`, prints a header, one block per simulated day and the final formatted report, and exits with code 0. A `--days` value below 1 shall produce a usage error with exit code 2, and `--help` shall list both options.	Functional	Medium	Instructor / Evaluator; Supply Chain Analyst	**AC-SCM-F-043:** `main(["--days", "3", "--seed", "42"])` returns 0 and its output contains `Day 1:`, `Day 3:` and `=== Supply Chain Report: Day 3 ===`. Running `python -m scm_sim --days 10 --seed 42` as a subprocess exits with code 0 and prints `=== Supply Chain Report: Day 10 ===`. `--days 0` exits with code 2. **Test: TC-SIM-02**	Function `main(argv)`, helper `describe_day`. Requires `PYTHONPATH=src` or running from `src/`. The demo has 2 suppliers, 2 warehouses, 3 SKUs and 14 scheduled orders.

5. Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and performance baselines of the Supply Chain Management Simulator. All measurements are established against the Python 3.11 runtime environment.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
SCM-NF-001	The system shall execute a 365-day run of the baseline demo scenario in s, and the 10-day command-line demo (including Python interpreter startup and shutdown) in s on a standard laptop with Python 3.11.	Performance	Medium	AC-SCM-NF-001: Execution time of `build_demo().run(365)` s (measured baseline ~0.02 s) and CLI invocation `python -m scm_sim --days 10 --seed 42` s (measured baseline ~0.10 s) across 3 consecutive runs. Test: TC-PERF-01
SCM-NF-002	The system shall produce identical daily reports and final summary output on every run and host platform when supplied with the identical random seed and simulation scenario configuration.	Reliability / Reproducibility	High	AC-SCM-NF-002: Two separate simulation runs with `--seed 42` generate 100% byte-for-byte identical daily logs and analytics reports; runs with different seeds (e.g., `--seed 7`) exhibit deterministic variations matching the pseudorandom sequence. Test: TC-REP-01
SCM-NF-003	The system test suite shall maintain line code coverage across the entire `scm_sim` package, measured automatically by `pytest-cov` on every GitHub Actions CI run; all tests must pass before merging any pull request.	Maintainability	High	AC-SCM-NF-003: Automated execution of `pytest -v --cov=scm_sim` reports statement coverage (measured baseline 97%) and zero failing tests on GitHub Actions ubuntu-latest. Test: TC-COV-01
SCM-NF-004	The system shall run across Windows, Linux, and macOS platforms on Python using solely the Python standard library (`dataclasses`, `enum`, `random`, `argparse`) at runtime, requiring third-party dependencies (`pytest`, `pytest-cov`) strictly for development and testing.	Portability	Medium	AC-SCM-NF-004: Clean environment verification: simulator runs and produces reports on Windows 11 and Ubuntu without any `pip install` runtime packages; `requirements.txt` contains only test dependencies. Test: TC-COV-01
SCM-NF-005	The system shall provide an intuitive command-line interface enabling a new user to execute the simulation scenario using a single documented command (`python -m scm_sim`), provide self-documenting option summaries via `--help`, and present clean report formatting displaying explicit item counts and printing “No issues” for empty report categories.	Usability	Medium	AC-SCM-NF-005: Executing `python -m scm_sim --help` outputs comprehensive CLI usage and parameter options; running the default demo prints formatted sections with counts and “No issues” for sections with zero anomalies; invalid inputs like `--days 0` terminate with exit code 2. Test: TC-SIM-02

5.1 Security Requirements

5.1.1 Security Objectives

The Supply Chain Management Simulator implements defense-in-depth principles appropriate for an in-memory discrete-event simulation engine and its development pipeline:

1. **SCM-SO-1 (Integrity of Simulation Data):** The system shall guarantee that all inventory, customer order, supplier, and shipment states remain strictly valid throughout the simulation lifecycle. The system state shall never transition into corrupted or physically impossible states (such as negative stock, exceeded warehouse capacity, illegitimate status regressions, or unvetted entity generation), even when supplied with invalid, malicious, or out-of-order input sequences.
2. **SCM-SO-2 (Integrity and Traceability of the Codebase):** The project shall ensure that only peer-reviewed, automatically verified, and auditable code is integrated into the production branch (`main`). Every modification must trace to a specific team member, a feature branch, and an approved Pull Request, preventing unauthorized code injection or regressions.

5.1.2 Security Requirements Specification

Req ID	Requirement (The system shall…)	Type	Priority	Acceptance criteria / Test case ref
SCM-SR-001	The system shall validate all input parameters for public operations (including order/shipment `quantity > 0`, supplier `lead_time_days >= 0`, supplier `reliability` in [0.0, 1.0], warehouse `capacity > 0`, non-negative `StockPolicy` thresholds with `max_level >= reorder_point`, and scheduled order `day > current_day`) and raise a `ValueError` before mutating internal state.	Security	High	AC-SCM-SR-001: Invoking domain methods with non-positive quantities, out-of-bounds reliabilities, negative lead times, or invalid stock policy thresholds immediately raises `ValueError` and leaves simulation state unmodified. Test: TC-SEC-01
SCM-SR-002	The system shall prevent warehouse stock levels from falling below zero or exceeding registered warehouse capacity; any operation violating these invariants shall raise `InsufficientStockError` or `CapacityExceededError` respectively, leaving inventory state unchanged.	Security	High	AC-SCM-SR-002: Attempting to remove more units than available raises `InsufficientStockError` with stock level unchanged; adding stock that would cause total warehouse units to exceed registered capacity raises `CapacityExceededError` with stock level unchanged. Test: TC-SEC-01
SCM-SR-003	The system shall guard order lifecycle status transitions such that orders in status `FULFILLED` cannot be cancelled and neither `FULFILLED` nor `CANCELLED` orders can be re-processed, re-fulfilled, or double-credited.	Security	High	AC-SCM-SR-003: Calling `OrderManager.cancel()` on an order with status `FULFILLED` raises `ValueError`; calling `try_fulfil()` on an already `FULFILLED` or `CANCELLED` order returns `False` without deducting inventory or modifying order attributes. Test: TC-SEC-01
SCM-SR-004	The system shall reject references to unknown suppliers, warehouses, or order IDs with a `KeyError`, preventing implicit entity generation or silent corruption of simulation state.	Security	High	AC-SCM-SR-004: Passing an unregistered supplier ID to `dispatch()`, an unregistered warehouse ID to `schedule_order()`, or a non-existent order ID to `get()` or `cancel()` raises `KeyError` with no state mutation. Test: TC-SEC-01
SCM-SR-005	The repository and collaboration platform shall enforce branch protection on `main` requiring a pull request, at least one code review approval from a non-author teammate, passing status checks for the automated test CI workflow (`test`), up-to-date branch status, and blocking of force pushes and branch deletions.	Security	High	AC-SCM-SR-005: Direct `git push` to `main` is rejected by GitHub branch protection rules; pull requests cannot be merged if the `test` CI job fails or if zero teammate approvals are recorded. Test: TC-SEC-02
SCM-SR-006	The repository and runtime codebase shall contain zero secrets, API keys, or credentials, make zero runtime network communications, and require zero third-party dependencies during runtime execution (relying strictly on the Python standard library).	Security	Medium	AC-SCM-SR-006: Static scan of repository files detects no credentials or tokens; `requirements.txt` specifies only testing utilities (`pytest`, `pytest-cov`); application executes without outbound network activity. Test: TC-SEC-02
SCM-SR-007	The system shall ensure that data returned to callers (including inventory snapshots via `snapshot()` and analytics report structures) are independent copies, such that external mutation of returned data structures does not alter internal state.	Security	Medium	AC-SCM-SR-007: Modifying the dictionary returned by `Inventory.snapshot()` or altering lists within reports generated by `build_report()` has no impact on internal `_stock` or simulation state. Test: TC-SEC-01

6. Quality Attributes & Acceptance Tests

This section establishes the quality standards, project exit criteria, and individual acceptance test suites required for system verification before release.

6.1 Exit Criteria

The simulator software and its deliverables are considered ready for acceptance when all of the following conditions are satisfied: 1. **100% Test Case Execution:** All planned functional and non-functional test cases (TC-INV-01 through TC-REP-01) are executed. 2. **Zero Defect Tolerance:** Zero open High or Critical severity defects exist in the codebase. 3. **Automated Test Pass Rate:** 100% pass rate achieved across all automated test cases in the test suite (73/73 tests passing). 4. **Code Coverage Threshold:** Line coverage across `src/scm_sim/` reaches or exceeds 90%, verified by `pytest-cov` (current baseline: 97%). 5. **CI Pipeline Stability:** GitHub Actions CI job `test` passes with a green check on every pull request targeting `main`. 6. **Peer Review Governance:** Every pull request receives at least one formal review and approval from a designated peer reviewer prior to merging.

6.2 Acceptance Test Suites

Acceptance Test Suite	Scope & Target Requirements	Measurable Acceptance Test
Inventory Acceptance Suite	Stock level adjustments, warehouse capacity limits, per-item policy evaluation, reorder suggestions, and state snapshots (`SCM-F-001`–`SCM-F-004`).	Executing `pytest tests/test_inventory.py` passes all unit tests, confirming that negative additions and capacity breaches raise appropriate exceptions (`ValueError`, `CapacityExceededError`) while leaving inventory state intact.
Orders Acceptance Suite	Customer order creation, sequential ID assignment, full-quantity fulfillment, FIFO backorder retries, and guarded cancellation (`SCM-F-010`–`SCM-F-013`).	Executing `pytest tests/test_orders.py` passes all unit tests, confirming that invalid order quantities () raise `ValueError`, fulfilled orders cannot be cancelled, and `process_open()` strictly fulfills oldest orders first.
Shipments Acceptance Suite	Supplier registry, lead-time tracking, probabilistic delay injection, arrival processing, and late shipment detection (`SCM-F-020`–`SCM-F-023`).	Executing `pytest tests/test_shipments.py` passes all unit tests, confirming that zero-reliability suppliers incur 1–3 days delay while reliability 1.0 suppliers arrive strictly on time.
Analytics Acceptance Suite	Detection of delayed shipments, stockouts, low stock, excess inventory, fill rate computation, and report formatting (`SCM-F-030`–`SCM-F-033`).	Executing `pytest tests/test_analytics.py` passes all unit tests, confirming that fill rate accurately discounts cancelled orders and formatted reports display explicit counts with “No issues” for empty sections.
Simulation & CLI Acceptance Suite	End-to-end scenario orchestration, 5-step daily simulation loop, automatic replenishment, and CLI interface (`SCM-F-040`–`SCM-F-043`).	Executing `pytest tests/test_simulator.py` passes all integration and system tests, and running `python -m scm_sim --days 10 --seed 42` returns exit code 0 while producing the complete daily log and summary report.
Performance Acceptance Suite	Runtime execution efficiency and benchmark thresholds (`SCM-NF-001`).	Running `build_demo().run(365)` completes in s (measured: ~0.02 s), and running the 10-day CLI demo completes in s (measured: ~0.10 s) across 3 consecutive trials on Python 3.11.
Reproducibility Acceptance Suite	Deterministic simulation execution across runs and platforms (`SCM-NF-002`).	Two executions of `python -m scm_sim --days 10 --seed 42` produce 100% identical outputs and analytics metrics; running with `--seed 7` produces a deterministically distinct set of delay and stockout events.

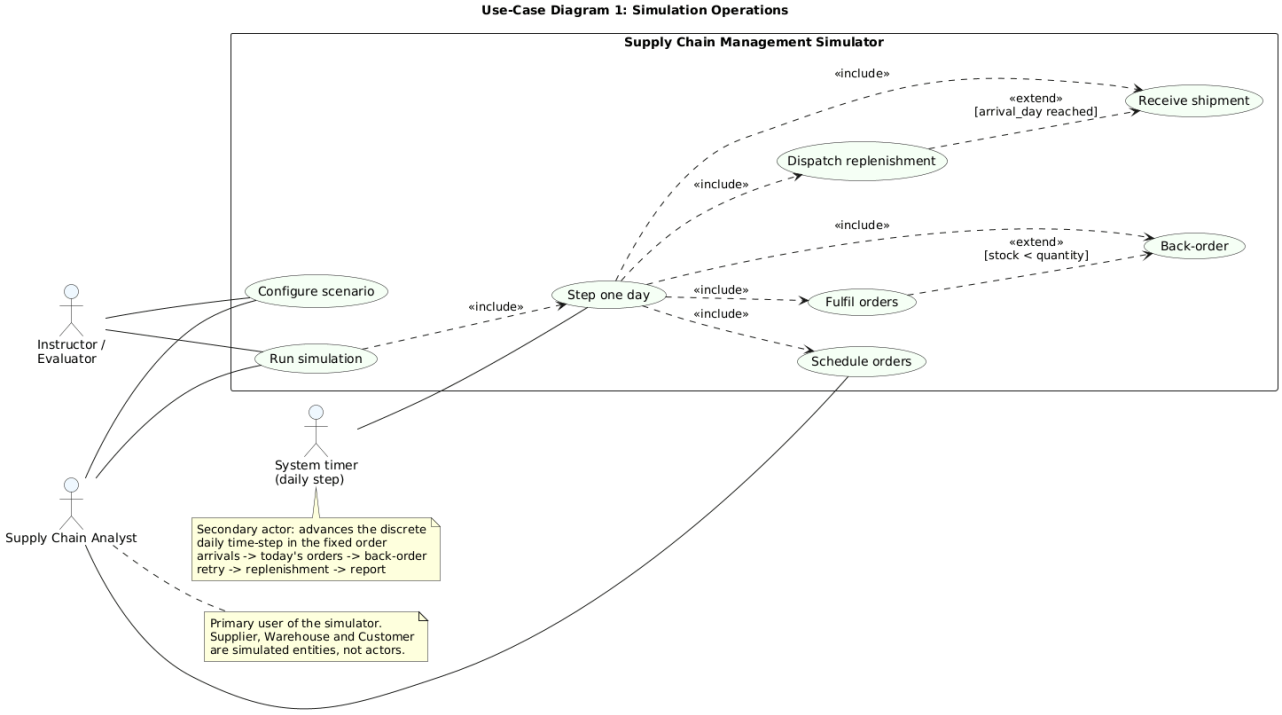
Acceptance Test Suite	Scope & Target Requirements	Measurable Acceptance Test
Security & Integrity Acceptance Suite	Input parameter validation, state invariants, secret avoidance, and branch protection (`SCM-SO-1`, `SCM-SO-2`, `SCM-SR-001`–`SCM-SR-007`).	Direct pushes to `main` are blocked by GitHub branch protection rules; invalid state mutations raise exceptions leaving memory untouched; no credentials or runtime third-party packages exist in the repository.

7. System Models

7.1 UML Use-Case Diagrams

This subsection models the externally observable behaviour of the Supply Chain Management Simulator from the point of view of the human and automated participants defined in SRS §2.3. Two use-case diagrams are provided: **Diagram 1 (Simulation operations)** describes how a run is configured and advanced, and **Diagram 2 (Monitoring & quality)** describes how the results of a run are inspected and how the development pipeline is exercised. Both diagrams use only the actors and use cases defined below, and both are accompanied by an actor table and a use-case traceability table.

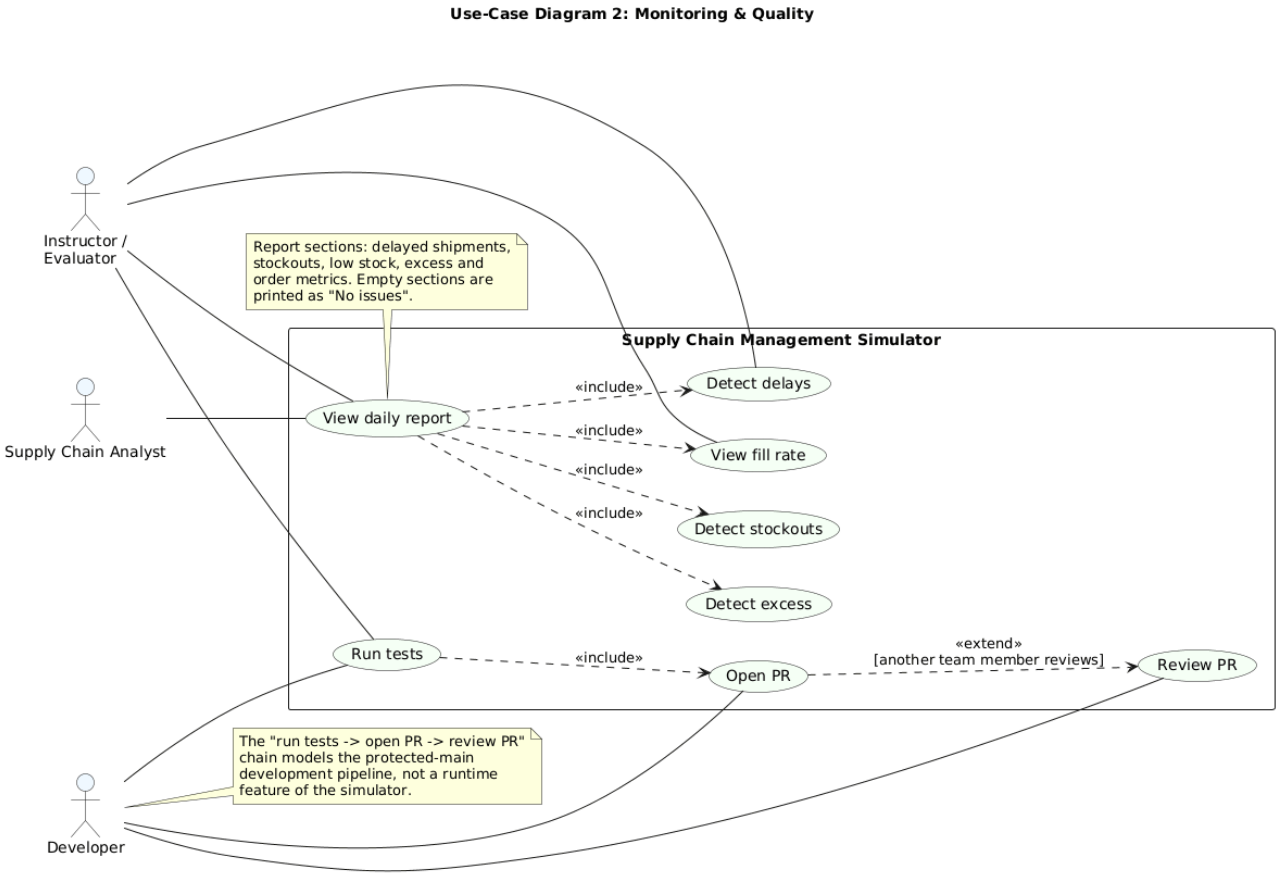
The following diagram presents the *Simulation operations* view of the system.



Use-case diagram: operations

Diagram 1 — Simulation operations. The Supply Chain Analyst configures a scenario (suppliers, warehouses, per-item policies, initial stock), schedules customer orders for future days and starts a run. The **System timer** is modelled as a secondary actor that represents the discrete daily time-step: once a run is active it triggers *Step one day*, which internally includes *Receive shipment*, *Fulfil orders*, *Back-order* and *Dispatch replenishment* in the fixed order defined by SCM-F-041. *Back-order* extends *Fulfil orders* whenever the available stock is less than the requested quantity (SCM-F-011), and *Receive shipment* extends *Dispatch replenishment* once the shipment's arrival day is reached (SCM-F-023). The Instructor / Evaluator configures and runs scenarios to assess the system. Supplier, Warehouse and Customer are simulated entities held in the domain state; they are **not** actors.

The following diagram presents the *Monitoring & quality* view of the system.



Use-case diagram: monitoring

Diagram 2 — Monitoring & quality. The Supply Chain Analyst *View daily report*, which includes the four detection and metric use cases *Detect delays* (SCM-F-030), *Detect stockouts*, *Detect excess* (both SCM-F-031) and *View fill rate* (SCM-F-032); the report text itself is produced by SCM-F-033. The Instructor / Evaluator reads the same reports and the aggregate metrics to assess whether the simulator behaves as specified. The Developer, a team member rather than an end user, *Run tests*, *Open PR* and *Review PR*; these three use cases model the protected-`main` development pipeline of SCM-SO-2 and SCM-SR-005 and are not runtime features of the simulator.

7.1.1 Actors

Actor	Type	Description	Use cases participated in
Supply Chain Analyst	Primary (human)	The end user of the simulator. Configures the supply network scenario, schedules customer demand, runs the simulation, and reads the daily analytics reports to find delays, stockouts, low stock, excess inventory and fill-rate shortfalls.	Configure scenario; Schedule orders; Run simulation; View daily report; Detect delays; Detect stockouts; Detect excess; View fill rate
System timer	Secondary (automated)	The discrete daily time-step of the simulation engine. Once a run is started it triggers one simulated day at a time, which drives shipment arrivals, order fulfilment, back-order retries and automatic replenishment in a fixed order.	Step one day; Fulfil orders; Back-order; Dispatch replenishment; Receive shipment
Instructor / Evaluator	Primary (human)	Runs the command-line demo and the test suite to assess the system against the problem statement and the requirements, and inspects the resulting reports and metrics.	Configure scenario; Run simulation; View daily report; Detect delays; View fill rate; Run tests
Developer	Supporting (human)	A member of the project team who extends a module, runs the automated test suite, and proposes reviewed changes to the repository through pull requests.	Run tests; Open PR; Review PR

Actor	Type	Description	Use cases participated in
Supplier *(not an actor)*	Simulated entity	External supplier entity modelled by the Shipment Manager. It receives no input from a human and produces no independent behaviour.	–
Warehouse *(not an actor)*	Simulated entity	Storage location modelled by the Inventory component.	–
Customer *(not an actor)*	Simulated entity	Origin of demand modelled by the Order Manager; customers only issue orders that the Analyst schedules.	–

7.1.2 Use Cases — Simulation Operations

Use case	Description	Primary / secondary actor	Requirements exercised
Configure scenario	Register suppliers with lead time and reliability, register warehouses with a capacity, attach a per-item stock policy to a supplier, and load the initial stock of every tracked item.	Supply Chain Analyst; Instructor / Evaluator	SCM-F-020, SCM-F-040
Schedule orders	Place customer orders against a warehouse and SKU for a future simulation day; past days, non-positive quantities and unknown warehouses are rejected.	Supply Chain Analyst	SCM-F-010, SCM-F-040
Run simulation	Execute the configured scenario for a given number of days and print the day-by-day log together with the final report; exit code 0 on success.	Supply Chain Analyst; Instructor / Evaluator	SCM-F-041, SCM-F-043
Step one day	Advance the simulation by exactly one simulated day, invoking shipment arrivals, today's orders, back-order retry, replenishment and report construction in that fixed order, and retain the resulting report.	System timer	SCM-F-041
Fulfil orders	Attempt each open order against available stock; on success the stock is reduced, the status becomes `FULFILLED` and the fulfilment day is recorded. There is no partial fulfilment.	System timer	SCM-F-011, SCM-F-012
Back-order	Mark an order `BACKORDERED` when stock is below the requested quantity, leaving stock unchanged, so that it is retried oldest-first on a later day.	System timer	SCM-F-011, SCM-F-012
Dispatch replenishment	Dispatch exactly one replenishment per item that is at or below its reorder point and has nothing already in transit, trimmed so that on-hand plus in-transit quantity never exceeds the warehouse capacity.	System timer	SCM-F-042, SCM-F-003
Receive shipment	On and after a shipment's arrival day, add its quantity to the destination warehouse stock, set the status to `DELIVERED` and record the actual day.	System timer	SCM-F-023

7.1.3 Use Cases — Monitoring & Quality

Use case	Description	Primary / secondary actor	Requirements exercised
View daily report	Read the report for the current	Supply Chain	SCM-F-033, SCM-F-041, SCM-NF-

Use case	Description	Primary / secondary actor	Requirements exercised
	day, which shows the day number, delayed shipments, stockouts, low stock, excess inventory and order counts with a fill rate.	Analyst; Instructor / Evaluator	005
Detect delays	List every shipment that is later than its expected arrival day, with its number of days late, ordered worst first.	Supply Chain Analyst; Instructor / Evaluator	SCM-F-030
Detect stockouts	List every tracked item whose quantity is exactly zero, and every item whose quantity is at or below the reorder point but above zero, so that the shortfall is visible before it becomes a stockout.	Supply Chain Analyst	SCM-F-031, SCM-F-003
Detect excess	List every tracked item above the maximum level together with the surplus quantity above that level, so that excess inventory and tied-up stock are visible.	Supply Chain Analyst	SCM-F-031
View fill rate	Read the counts of orders per status and the fill rate, defined as fulfilled orders divided by total orders excluding cancelled orders, rounded to two decimal places.	Supply Chain Analyst; Instructor / Evaluator	SCM-F-032
Run tests	Execute the automated pytest suite, with coverage measurement in CI, to confirm that every merged change is still correct.	Developer; Instructor / Evaluator	SCM-NF-003, SCM-NF-004
Open PR	Propose a change on a feature branch for merge into the protected `main` branch, accompanied by the passing `test` check.	Developer	SCM-SO-2, SCM-SR-005
Review PR	Inspect the files changed in a pull request against the requirements registry, leave at least one review comment, and approve so that the merge is permitted.	Developer	SCM-SO-2, SCM-SR-005

8. Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) records, for every requirement defined in this SRS, the section that specifies it, the module that implements it, the test case that verifies it, and the verification status. It is the primary index used during final acceptance to confirm that every requirement in SRS §4, §5 and §5.1 is implemented and verified.

Status codes

Code	Meaning
A	Accepted — the linked test case is automated, has executed, and passed in the CI `test` job on GitHub Actions (CI run #20, commit `b3702ef`, 11-10-2026).
N	Not yet verified — the test case is manual, or contains a manual step that has not been executed yet.

8.1 Functional Requirements

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SCM-F-001	Stock tracking per (warehouse, SKU) with	SRS §4.1; SAD §4.3.1	`scm_sim.inventory`	TC-INV-01	A	`InsufficientStockError` leaves stock

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
	validated add/remove		(Inventory)			unchanged.
SCM-F-002	Warehouse capacity with CapacityExceededError on breach	SRS §4.1; SAD §4.3.1	scm_sim.inventory (Inventory)	TC-INV-01	A	Unregistered warehouses have unlimited capacity.
SCM-F-003	Per-item stock policy; needs_reorder / is_excess	SRS §4.1; SAD §4.3.1	scm_sim.inventory (StockPolicy)	TC-INV-02	A	Defaults reorder_point 20, reorder_quantity 100, max_level 500. Analytics reuses the 20 / 500 defaults in SCM-F-031.
SCM-F-004	Sorted reorder suggestions and independent stock snapshot	SRS §4.1; SAD §4.3.1	scm_sim.inventory (Inventory)	TC-INV-02	A	snapshot() is the input to the analytics stock classifiers; satisfies SCM-SR-007.
SCM-F-010	Create customer orders with sequential ORD-0001… IDs	SRS §4.2; SAD §4.3.2	scm_sim.orders (OrderManager)	TC-ORD-01	A	Initial status PENDING.
SCM-F-011	Fulfil in full or back-order; no partial fulfilment	SRS §4.2; SAD §4.3.2	scm_sim.orders (OrderManager)	TC-ORD-01	A	Base data for the fill-rate metric in SCM-F-032.
SCM-F-012	Retry open orders oldest-first	SRS §4.2; SAD §4.3.2	scm_sim.orders (OrderManager)	TC-ORD-02	A	Ordering by created_day, then order_id.
SCM-F-013	Cancel, look up by ID, list by status	SRS §4.2; SAD §4.3.2	scm_sim.orders (OrderManager)	TC-ORD-02	A	Unknown ID raises KeyError (SCM-SR-004).
SCM-F-020	Register suppliers with lead time and reliability	SRS §4.3; SAD §4.3.3	scm_sim.shipments (ShipmentManager)	TC-SHP-01	A	Duplicate supplier IDs rejected.
SCM-F-021	Dispatch shipments with sequential SHP-0001… IDs	SRS §4.3; SAD §4.3.3	scm_sim.shipments (ShipmentManager)	TC-SHP-01	A	expected_day = day + lead_time_days.
SCM-F-022	Model stochastic delays from supplier reliability	SRS §4.3; SAD §4.3.3	scm_sim.shipments (ShipmentManager)	TC-SHP-02	A	Seeded PRNG keeps runs reproducible (SCM-NF-002).
SCM-F-023	Deliver arrivals and list shipments with days late > 0	SRS §4.3; SAD §4.3.3	scm_sim.shipments (ShipmentManager)	TC-SHP-02	A	Supplies expected_day / actual_day consumed by SCM-F-030.
SCM-F-030	Report delayed shipments with days late, worst first	SRS §4.4; SAD §4.3.4	scm_sim.analytics (find_delayed_shipments)	TC-ANA-01	A	Uses actual_day when delivered, otherwise current_day.
SCM-F-031	Report stockouts, low stock and excess with surplus	SRS §4.4; SAD §4.3.4	scm_sim.analytics (find_stockouts, find_low_stock, find_excess)	TC-ANA-01	A	Boundary values 0 / 20 / 21 and 500 / 501 verified.
SCM-F-032	Order metrics per status and fill rate (2 dp)	SRS §4.4; SAD §4.3.4	scm_sim.analytics (order_metrics)	TC-ANA-02	A	Cancelled orders excluded from the denominator.

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SCM-F-033	Build the daily report and format it as readable text	SRS §4.4; SAD §4.3.4	`scm_sim.analytics` (`build_report`, `format_report`)	TC-ANA-02	A	Empty sections print “No issues”; satisfies the reporting rule of SCM-NF-005.
SCM-F-040	Configure a scenario	SRS §4.5; SAD §3.4.1	`scm_sim.simulator` (`Simulation`)	TC-SIM-02	A	Unknown warehouses and past days rejected.
SCM-F-041	Advance one day in the fixed order and keep every report	SRS §4.5; SAD §4.1	`scm_sim.simulator` (`Simulation`)	TC-SIM-01	A	Report construction is step 5 and calls `build_report` (SCM-F-033).
SCM-F-042	Automatic replenishment, trimmed to capacity	SRS §4.5; SAD §4.1	`scm_sim.simulator` (`Simulation`)	TC-SIM-01	A	Exactly one dispatch per item while nothing is in transit.
SCM-F-043	Command-line interface with exit code 0	SRS §4.5; SAD §4.5	`scm_sim.__main__`	TC-SIM-02	A	`--days < 1` produces a usage error with exit code 2.

8.2 Non-Functional Requirements

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SCM-NF-001	365-day run and 10-day CLI demo each complete in s	SRS §5; SAD §3.1	`scm_sim.simulator`, `scm_sim.__main__`	TC-PERF-01	N	Manual timing test with `time` / `Measure-Command`, 3 runs each; to be executed and recorded by 20-10-2026. Measured baselines recorded in SRS §5 are ~0.02 s and ~0.10 s.
SCM-NF-002	Same seed and scenario give identical output on every run and machine	SRS §5; SAD §3.6	`scm_sim.simulator` (`random.Random(seed)`)	TC-REP-01	N	Mixed case: the automated part is accepted (A) — `tests/test_simulator.py::test_same_seed_gives_identical_reports` passed in CI run #20. The manual part (running the CLI twice with `--seed 42` and comparing output, then with `--seed 7`) has not been executed, so the overall status is N until it is.
SCM-NF-003	line coverage of `scm_sim`, all tests pass before merge	SRS §5; SAD §3.6	`scm_sim` package (all modules)	TC-COV-01	N	Mixed case: the CI-measured part is accepted (A) — `pytest -v --cov=scm_sim` reported 73 passed and 97% coverage in CI run #20, commit `b3702ef`. Coverage is re-measured on every push to `main` and every PR, so it is continuously satisfied rather than verified once. Status N because the manual portability execution of the same test case is outstanding.
SCM-NF-004	Runs on Python 3.11 on Windows, Linux and macOS, standard library only at runtime	SRS §5; SAD §3.6	`scm_sim` package; `requirements.txt`	TC-COV-01	N	Same mixed case as SCM-NF-003: automated on `ubuntu-latest` with Python 3.11 (A); the manual runs on Windows 11 and a

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
						local Linux laptop have not been executed yet. `requirements.txt` holds test-only dependencies, satisfying SCM-SR-006.
SCM-NF-005	One documented command to run the demo; counts and "No issues" in every section; `--help` lists all options	SRS §5; SAD §4.5	`scm_sim.__main__`; `scm_sim.analytics` (`format_report`)	TC-SIM-02	A	The empty-section rule is implemented by `format_report` (SCM-F-033).

8.3 Security Objectives and Security Requirements

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SCM-SO-1	**Objective:** integrity of simulation data — stock, order and shipment states can never become invalid, even with bad input	SRS §5.1.1; SAD §3.9	All domain modules (`Inventory`, `OrderManager`, `ShipmentManager`)	TC-SEC-01	–	Objective, not directly testable; verified through the SRs below, which are all accepted.
SCM-SO-2	**Objective:** integrity and traceability of the codebase — only reviewed, tested changes reach `main`, each traceable to an author and a PR	SRS §5.1.1; SAD §3.9	Repository workflow; `.github/workflows/ci.yml`	TC-SEC-02	N	Objective, not directly testable; enforced procedurally by SCM-SR-005 and SCM-SR-006. The manual branch-protection inspection of TC-SEC-02 is outstanding.
SCM-SR-001	Validate input on every public operation and raise `ValueError` before changing state	SRS §5.1.2; SAD §3.9	All domain modules; `scm_sim.__main__`	TC-SEC-01	A	Analytics performs no validation and raises nothing, because it is read-only and holds no state.
SCM-SR-002	Stock never goes below 0 or above a registered capacity; violations raise and leave state unchanged	SRS §5.1.2; SAD §3.9	`scm_sim.inventory`	TC-SEC-01	A	`InsufficientStockError` / `CapacityExceededError`.
SCM-SR-003	Guarded order status transitions	SRS §5.1.2; SAD §3.9	`scm_sim.orders`	TC-SEC-01	A	`FULFILLED` cannot be cancelled; `FULFILLED` and `CANCELLED` are never re-processed.
SCM-SR-004	Unknown supplier, warehouse or order references raise `KeyError`	SRS §5.1.2; SAD §3.9	`scm_sim.orders`, `scm_sim.shipments`, `scm_sim.simulator`	TC-SEC-01	A	Nothing is created implicitly.
SCM-SR-005	`main` is protected: PR required, one non-author approval, `test` check passing, up-to-date branch, no force pushes or deletions	SRS §5.1.2; SAD §3.9	GitHub branch protection; CI `test` job	TC-SEC-02	N	Manual process test: direct push to `main` must be rejected, a PR without approval must be unmergeable, and Settings → Branches must show the rules. Not yet executed.
SCM-SR-006	No secrets in the repository; no network calls and no runtime third-party dependencies	SRS §5.1.2; SAD §3.9	Repository; `requirements.txt`	TC-SEC-02	N	Verified manually by searching the repository for tokens and checking that `requirements.txt` lists only test dependencies. Not yet executed.
SCM-SR-007	Data returned to callers is a copy; mutating it cannot	SRS §5.1.2; SAD §3.9	`scm_sim.inventory`	TC-SEC-01	A	`Inventory.snapshot()` returns an independent

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
	change internal state		(`snapshot`); `scm_sim.analytics` (`build_report`)			copy; `build_report` builds a fresh dictionary that the caller may retain.

8.4 Coverage Summary

Requirement set	Total	Accepted (A)	Not yet verified (N)	Objective (–)
Functional requirements (§4)	20	20	0	0
Non-functional requirements (§5)	5	1	4	0
Security requirements (§5.1.2)	7	5	2	0
Security objectives (§5.1.1)	0	0	0	2
Total	**32**	**26**	**6**	**2**

All 20 functional requirements are accepted. The six requirements carrying status N are all blocked on manual execution rather than on missing verification work: TC-PERF-01 (SCM-NF-001), the manual half of TC-REP-01 (SCM-NF-002), the manual portability half of TC-COV-01 (SCM-NF-003 and SCM-NF-004), and both requirements of TC-SEC-02 (SCM-SR-005 and SCM-SR-006). These are scheduled for completion by 20-10-2026 per Test Plan §7, after which their status moves to A and the accepted count reaches 32 of 32.